

Übung 5: Records

1. Das Szenario

Wir entwickeln ein Verwaltungssystem für Patentanmeldungen. Da Patente nach der Einreichung (*Filing*) als Dokumente unveränderlich sind, eignen sich **Java Records** perfekt für die Datenhaltung. Wir modellieren Anmeldungen, validieren diese und implementieren Statusübergänge.

Lernziele:

- Definieren von Java Records
 - Nutzung des *Compact Constructor* zur Validierung
 - Implementieren von Business-Logik in Records
 - Umgang mit Unveränderlichkeit (*Wither*-Pattern)
-

2. Die Aufgaben

Schritt 1: Grundstruktur

Erstelle zwei Records, um die Datenstruktur abzubilden. Verzichte zunächst auf Getter/Setter oder `equals/hashCode` (diese bringt der Record automatisch mit).

1. **Erfinder Record:**
 - `name` (String)
 - `email` (String)
2. **PatentAnmeldung Record:**
 - `id` (String, z.B. "DE-12345")
 - `titel` (String)
 - `einreichungsDatum` (LocalDate)
 - `hauptErfinder` (Typ: **Erfinder**)
 - `status` (Enum: EINGEREICHT, GEPRUEFT, ERTEILT, ABGELEHNT)

Schritt 2: Validierung (Kompakter Konstruktor)

Records erlauben einen *Compact Constructor* ohne Parameterliste zur Validierung. Erweitere deine Records:

- **Im Erfinder:** Stelle sicher, dass `name` nicht null und nicht leer ist. Falls doch, wirf eine `IllegalArgumentException`.
- **In PatentAnmeldung:** Das `einreichungsDatum` darf nicht in der Zukunft liegen.

Schritt 3: Methoden

Records können Methoden enthalten. Füge der **PatentAnmeldung** folgende Logik hinzu:

- Schreibe eine Methode `boolean istVeroeffentlichbar()`.
- **Logik:** Eine Anmeldung ist nur veröffentlichbar, wenn der Status `ERTEILT` ist **und** das Einreichungsdatum älter als 18 Monate ist.

Schritt 4: Unveränderlichkeit vs. Zustandsänderung

Records sind *immutable*. Wir können den Status nicht einfach per Setter ändern.

- Implementiere in **PatentAnmeldung** eine Methode `mitStatus(Status neuerStatus)`.
- Diese Methode soll eine **neue Instanz** von **PatentAnmeldung** zurückgeben, die alle alten Daten enthält, aber den neuen Status besitzt.